

SwapAdvisor: Push Deep Learning Beyond the GPU Memory Limit via Smart Swapping

Gustavo B. P. Silva¹, Michael P. C. Silva¹, Rafael S. Melo¹

¹Departamento de Ciência da Computação – Universidade Federal de Catalão (UFCAT)

{gustavo.silva, michael.silva}@discente.ufcat.edu.br,
rafinhalelograma@hotmail.com

Resumo. Breve resumo do tema do trabalho.

1. Introdução

A questão de pesquisa do artigo é como superar a limitação de memória da GPU para treinar e inferir modelos de redes neurais profundas (DNNs) maiores e mais precisos. A hipótese é que a troca de dados entre a memória da GPU e da CPU pode resolver essa limitação sem comprometer a precisão do modelo.

A lacuna de pesquisa identificada é a incapacidade dos métodos existentes de troca de dados de suportar modelos grandes de DNNs de forma eficiente. Os métodos atuais lidam apenas com certos modelos e não alcançam um desempenho satisfatório. O artigo propõe resolver essa limitação através de uma abordagem otimizada de troca de dados, o SwapAdvisor.

Nos últimos anos, a comunidade de aprendizado profundo tem utilizado redes neurais profundas (DNNs) maiores para melhorar a precisão em tarefas complexas, com um aumento constante no número de parâmetros das redes, que dobra a cada 2,4 anos desde os anos 80, impulsionado por melhorias em hardware e dados (Goodfellow et al. 2016). No entanto, a memória da GPU limita o tamanho de modelos DNN, com abordagens existentes buscando reduzir o consumo de memória através de técnicas como menor precisão em pontos flutuantes, quantização e esparsificação, mas estas podem afetar a precisão e exigem ajustes complexos de hiperparâmetros (Gupta et al. 2015, Judd et al. 2016). Outra solução é recalcular dados intermediários, mas não suporta modelos grandes (Chen et al. 2016, Martens e Sutskever 2012).

Uma abordagem promissora é a troca de dados entre a memória da GPU e da CPU, aproveitando a maior capacidade e custo mais baixo da memória da CPU, além da melhoria na comunicação entre GPU e CPU com tecnologias como PCIe 5.0 e NVLink (Le et al. 2018, Meng et al. 2017, Rhu et al. 2016, Wang et al. 2018). No entanto, muitas abordagens atuais utilizam informações limitadas do gráfico de fluxo de dados para planejar a troca, prejudicando o desempenho (Yu et al. 2018, Le et al. 2018).

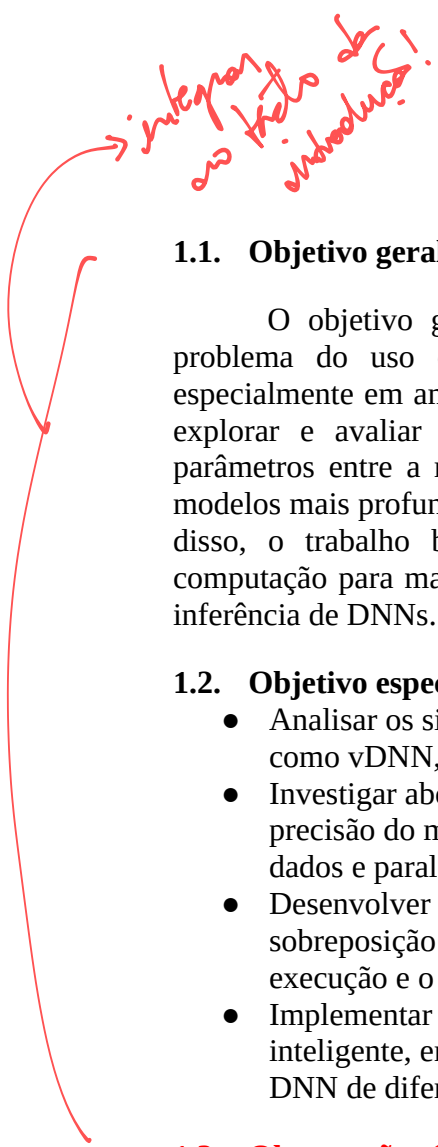
Neste artigo, os pesquisadores Huang, Jin e Li (2020), propuseram o SwapAdvisor, que se trata de um sistema para otimizar a troca de dados em DNNs, planejando precisamente o que e quando trocar para maximizar a sobreposição de computação e comunicação. Ele usa um algoritmo genético para otimizar a alocação de memória e o agendamento de operadores, permitindo o treinamento de modelos até 12 vezes o limite da memória da GPU, com taxas de transferência entre 53% e 99% da linha de base ideal. O SwapAdvisor também é útil para inferência, permitindo a utilização de múltiplos modelos em uma única GPU com latência reduzida em até

quatro vezes em comparação com a abordagem de compartilhamento de tempo (Le et al. 2018, Meng et al. 2017).

O SwapAdvisor é o primeiro sistema geral de troca para modelos DNN grandes, mas possui limitações, como a necessidade de um gráfico de fluxo de dados estático e a limitação a uma única GPU, exigindo mais pesquisas para superar essas restrições.

Restrições aqui!

} H. (30%) : 30%
L. (30%) : 30%
E. I. (40%) : 30%



1.1. Objetivo geral

O objetivo geral deste trabalho é investigar e propor uma solução para o problema do uso eficiente da memória em Redes Neurais Profundas (DNNs), especialmente em ambientes com restrições de memória, como GPUs. A pesquisa visa explorar e avaliar diferentes abordagens para a troca (swapping) de tensores e parâmetros entre a memória da GPU e da CPU, de modo a permitir a execução de modelos mais profundos e maiores, sem comprometer o desempenho e a precisão. Além disso, o trabalho busca explorar estratégias de sobreposição de comunicação e computação para maximizar a utilização da GPU durante o processo de treinamento e inferência de DNNs.

1.2. Objetivo específico

- Analisar os sistemas existentes de troca de tensores e parâmetros em DNNs, como vDNN, TFLMS e SuperNeurons, e identificar suas limitações.
- Investigar abordagens alternativas para otimizar o uso de memória sem afetar a precisão do modelo, incluindo técnicas como recomputação, paralelismo de dados e paralelismo de modelo.
- Desenvolver uma solução que combine técnicas de troca de tensores e sobreposição de comunicação e computação, visando reduzir o tempo de execução e o consumo de memória sem prejudicar a performance do modelo.
- Implementar e avaliar a proposta de SwapAdvisor, um sistema de troca inteligente, em termos de eficiência de memória e desempenho em modelos DNN de diferentes profundidades e larguras.

1.3. Observações (ISSO É TEMPORÁRIO, APENAS PARA LEMBRARMOS)

Por se tratar de artigo acadêmico, teríamos que listar todos os autores para dar o devido crédito a cada um. No entanto, tem algumas referências que possuem muitos autores participantes no artigo original que estamos analisando, portanto, para reduzir texto, usaremos a seguinte terminologia:

- **Primeiro autor seguido de "et al."**: quando houver muitos autores, iremos listar apenas o primeiro autor seguido de "et al." (abreviação de "et alii", que significa "e outros" em latim). Por exemplo:
 - Em vez de [He, Zhang, Ren, e Sun 2016], podemos usar [He et al. 2016].
- **Número máximo de autores**: no modelo apresentado pelo professor, não foi dito em lugar algum a quantidade máxima de autores que pode colocar, portanto, iremos adicionar 2 autores no máximo, passou disso vamos usar "et al." para os autores restantes. Por exemplo:
 - [He, Zhang, Ren, and Sun 2016] pode ser reduzido para [He, Zhang, et al. 2016].
- **As referências**: consideramos copiar todas as referências do artigo original para darmos créditos, **mas iremos verificar ainda** se podemos excluir as referências que não forem utilizadas (citadas de forma direta ou indireta) para economizar espaço.

2. Metodologia

A metodologia empregada neste trabalho visa a integração de múltiplas abordagens de otimização para lidar com as limitações de memória ao executar Redes Neurais Profundas (DNNs) em GPUs. A principal inovação é o desenvolvimento de um sistema inteligente de troca de tensores e parâmetros entre a memória da GPU e da CPU, denominado *SwapAdvisor*, que também incorpora técnicas de sobreposição de comunicação e computação para maximizar a eficiência do uso da GPU.

2.1. Objetivo Geral da Metodologia

Desenvolver e implementar uma solução, o *SwapAdvisor*, que combina técnicas de troca de tensores e parâmetros com sobreposição de comunicação e computação, com o intuito de melhorar a utilização da memória em GPUs e permitir a execução eficiente de modelos DNN maiores e mais profundos, sem afetar a precisão e o desempenho geral.

2.2. Objetivos Específicos da Metodologia

- **Análise dos sistemas existentes:** O primeiro passo da metodologia consiste em revisar e entender os sistemas de troca de memória e paralelismo de modelos existentes, como vDNN, TFLMS e SuperNeurons, para identificar suas limitações e pontos fortes. Este estudo serve como base para a criação do *SwapAdvisor*, buscando a superação das limitações observadas em sistemas anteriores, como a incapacidade de suportar grandes parâmetros ou modelos RNNs.
- **Desenvolvimento do sistema *SwapAdvisor*:** A proposta de *SwapAdvisor* combina estratégias de troca de tensores com sobreposição de comunicação e computação, o que permite otimizar o uso de memória da GPU sem comprometer o tempo de execução. A troca de tensores é feita de forma inteligente, levando em consideração os tensores mais críticos e as iterações anteriores. Além disso, o sistema é projetado para suportar modelos DNN de diferentes tamanhos e complexidades, como redes convolucionais (CNNs) e redes recorrentes (RNNs).
- **Avaliação da abordagem *SwapAdvisor*:** A eficácia do *SwapAdvisor* foi avaliada em termos de redução do consumo de memória e aumento do desempenho (tempo de execução). A solução foi testada em diferentes modelos DNN, desde redes mais profundas e largas até RNNs com grandes parâmetros. A avaliação comparou o *SwapAdvisor* com outras abordagens existentes, como recomputação, paralelismo de dados e paralelismo de modelo, para determinar suas vantagens em termos de eficiência de memória e desempenho.
- **Sobreposição de comunicação e computação:** A metodologia também inclui a implementação de técnicas de sobreposição de comunicação e computação. Diferente de abordagens anteriores que dividem os kernels DNN de grão grosso em sub-kernels finos, que resultam em uma sobrecarga de desempenho, o *SwapAdvisor* utiliza uma abordagem de pré-busca, permitindo a sobreposição de comunicação e computação dentro de um gráfico de fluxo de dados de muitos kernels de grão grosso. Isso visa minimizar a latência e melhorar a utilização da GPU.

References

Referências

*certifique-se
que todos
tenham acesso!*

- CC. Huang, G. Jin, J. Li (2020) "SwapAdvisor: Push Deep Learning Beyond the GPU Memory Limit via Smart Swapping", In: 2020 ASPLOS '20, <https://dl.acm.org/doi/abs/10.1145/3373376.3378530>.
- Bastem, B., Unat, D., Zhang, W., Almgren, A., and Shalf, J. (2017) "Overlapping Data Transfers with Computation on GPU with Tiles", In: 2020,17 ASPLOS
- Chen, T., Xu, B., Zhang, C., and Guestrin, C. (2016) "Training deep nets with sublinear memory cost", arXiv preprint arXiv:1604.06174.
- Courbariaux, M., Bengio, Y., and David, J.-P. (2015) "BinaryConnect: Training Deep Neural Networks with Binary Weights during Propagations", In: Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2 (NIPS'15), MIT Press, pp. 3123–3131.
- Cui, H., Cipar, J., Ho, Q., Kim, J. K., Lee, S., Kumar, A., Wei, J., Dai, W., Ganger, G. R., Gibbons, P. B., and et al. (2014) "Exploiting Bounded Staleness to Speed up Big Data Analytics", In: Proceedings of the 2014 USENIX Conference on USENIX Annual Technical Conference (USENIX ATC'14), USENIX Association, pp. 37–48.
- Davis, L. (1991) Handbook of genetic algorithms.
- Dean, J., Corrado, G. S., Monga, R., Chen, K., Devin, M., Le, Q. V., Mao, M. Z., Ranzato, M. A., Senior, A., Tucker, P., and et al. (2012) "Large Scale Distributed Deep Networks", In: Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1 (NIPS'12), Curran Associates Inc., pp. 1223–1231.
- Gai, K., Qiu, M., and Zhao, H. (2016) "Cost-Aware Multimedia Data Allocation for Heterogeneous Memory Using Genetic Algorithm in Cloud Computing", IEEE Transactions on Cloud Computing, pp. 1–1.
- Goldberg, D. E., and Holland, J. H. (1988) "Genetic algorithms and machine learning", Machine Learning, 3(2).
- Gong, Y., Liu, L., Yang, M., and Bourdev, L. (2014) "Compressing deep convolutional networks using vector quantization", arXiv preprint arXiv:1412.6115.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016) Deep Learning, MIT Press. <http://www.deeplearningbook.org>.
- Gruslys, A., Munos, R., Danihelka, I., Lanctot, M., and Graves, A. (2016) "Memory-Efficient Backpropagation through Time", In: Proceedings of the 30th International Conference on Neural Information Processing Systems (NIPS'16), Curran Associates Inc., pp. 4132–4140.
- Gupta, S., Agrawal, A., Gopalakrishnan, K., and Narayanan, P. (2015) "Deep Learning with Limited Numerical Precision", In: Proceedings of the 32nd International Conference on International Conference on Machine Learning - Volume 37 (ICML'15), JMLR.org, pp. 1737–1746.
- Han, S., Liu, X., Mao, H., Pu, J., Pedram, A., Horowitz, M. A., and Dally, W. J. (2016) "EIE: Efficient Inference Engine on Compressed Deep Neural Network", SIGARCH Comput. Archit. News, 44(3), pp. 243–254.
- Han, S., Mao, H., and Dally, W. J. (2015) "Deep compression: Compressing deep

- neural networks with pruning, trained quantization and huffman coding”, arXiv preprint arXiv:1510.00149.
- Han, S., Pool, J., Tran, J., and Dally, W. J. (2015) “Learning Both Weights and Connections for Efficient Neural Networks”, In: Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 1 (NIPS’15), MIT Press, pp. 1135–1143.
- He, K., Zhang, X., Ren, S., and Sun, J. (2016) “Deep Residual Learning for Image Recognition”, In: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778.
- Hochreiter, S., and Schmidhuber, J. (1997) “Long Short-Term Memory”, Neural Computation, 9(8), pp. 1735–1780.
- Holland, J. H., et al. (1992) Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence, MIT Press.
- Hou, E. S. H., Ansari, N., and Ren, H. (1994) “A genetic algorithm for multiprocessor scheduling”, IEEE Transactions on Parallel and Distributed Systems, 5(2), pp. 113–120.
- Hubara, I., Courbariaux, M., Soudry, D., El-Yaniv, R., and Bengio, Y. (2016) “Binarized Neural Networks”, In: Proceedings of the 30th International Conference on Neural Information Processing Systems (NIPS’16), Curran Associates Inc., pp. 4114–4122.
- Jeong, E., Cho, S., Yu, G.-I., Jeong, J. S., Shin, D.-J., and Chun, B.-G. (2019) “JANUS: Fast and Flexible Deep Learning via Symbolic Graph Execution of Imperative Programs”, In: 16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19), USENIX Association, pp. 453–468.
- Jia, Z., Lin, S., Qi, C. R., and Aiken, A. (2018) “Exploring Hidden Dimensions in Parallelizing Convolutional Neural Networks”, In: International Conference on Machine Learning.
- Jia, Z., Zaharia, M., and Aiken, A. (2018) “Beyond data and model parallelism for deep neural networks”, arXiv preprint arXiv:1807.05358.
- Judd, P., Albericio, J., Hetherington, T., Aamodt, T. M., Jerger, N. E., and Moshovos, A. (2016) “Proteus: Exploiting Numerical Precision Variability in Deep Neural Networks”, In: Proceedings of the 2016 International Conference on Supercomputing (ICS’16), Association for Computing Machinery, Article 23.
- Krizhevsky, A. (2014) “One weird trick for parallelizing convolutional neural networks”, arXiv preprint arXiv:1404.5997.
- Le, T. D., Imai, H., Negishi, Y., and Kawachiya, K. (2018) “Tflms: Large model support in tensorflow by graph rewriting”, arXiv preprint arXiv:1807.02037.
- Li, M., Andersen, D. G., Park, J. W., Smola, A. J., Ahmed, A., Josifovski, V., Long, J., Shekita, E. J., and Su, B.-Y. (2014) “Scaling Distributed Machine Learning with the Parameter Server”, In: Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation (OSDI’14), USENIX Association, pp. 583–598.
- Man, K.-F., Tang, K. S., and Kwong, S. (2001) Genetic algorithms: concepts and

designs, Springer Science & Business Media.

- Martens, J., and Sutskever, I. (2012) “Training deep and recurrent networks with hessian-free optimization”, In: *Neural Networks: Tricks of the Trade*, Springer.
- Meng, C., Sun, M., Yang, J., Qiu, M., and Gu, Y. (2017) “Training deeper models by GPU memory optimization on TensorFlow”, In: *Proc. of ML Systems Workshop in NIPS*.
- Mirhoseini, A., Goldie, A., Pham, H., Steiner, B., Le, Q. V., and Dean, J. (2018) “A hierarchical model for device placement”.
- Mirhoseini, A., Pham, H., Le, Q. V., Steiner, B., Larsen, R., Zhou, Y., Kumar, N., Norouzi, M., Bengio, S., and Dean, J. (2017) “Device Placement Optimization with Reinforcement Learning”, In: *Proceedings of the 34th International Conference on Machine Learning - Volume 70 (ICML’17)*, JMLR.org, pp. 2430–2439.
- Ning, L., and Shen, X. (2019) “Deep Reuse: Streamline CNN Inference on the Fly via Coarse-Grained Computation Reuse”, In: *Proceedings of the ACM International Conference on Supercomputing (ICS’19)*, Association for Computing Machinery, pp. 438–448.